

Agentic AI Digital Twin

for Supply Chain Exception Management

Project plan, system architecture, and team execution guide

1. Problem statement

When a disruption hits a supply chain — a supplier delay, a demand spike, a shipment stuck in transit — human planners find out late, and by the time they react, part of the damage (stockout, missed delivery, excess cost) has already happened. Existing ERP systems show dashboards and threshold alerts, but they don't tell a planner what will happen next if they don't act, or which of several possible responses is best.

The data needed to predict the consequences already exists inside the system — it's just never simulated forward. This project closes that gap.

Why this matters

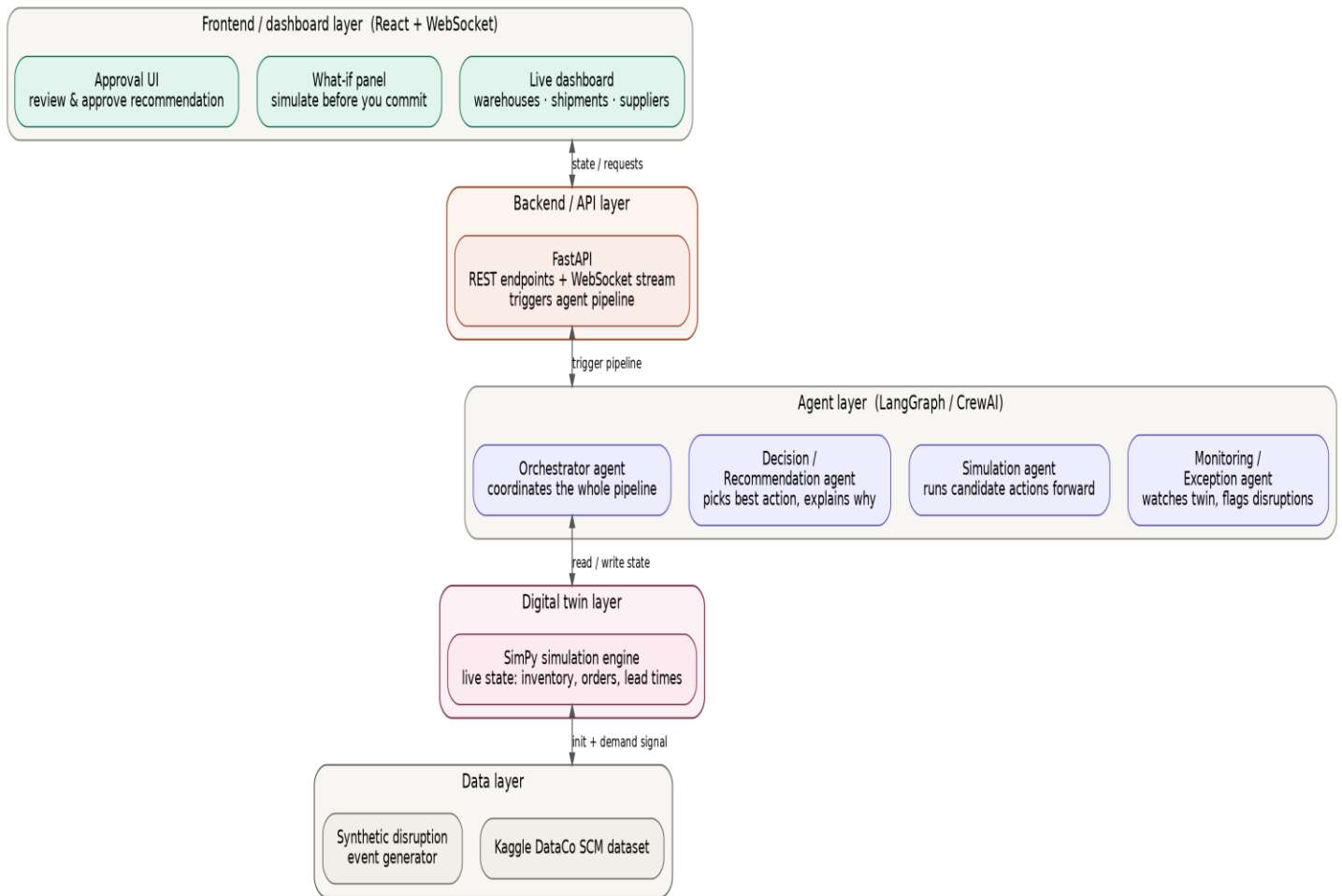
- Supply chains are interconnected: one delayed shipment cascades into stockouts, missed orders, and cost overruns across multiple downstream points — faster than a human can mentally trace.
- Rule-based ERP alerts (“stock below threshold”) report what already happened, not what's about to happen or what to do about it.
- Planners currently pull data from inventory, procurement, and logistics modules by hand, guess at impact, and decide on a fix — slow and inconsistent across planners.
- There's no safe way to test a response (reroute vs. reorder vs. wait) before committing to it in the real system.

2. Objectives

- Detect supply chain disruptions (delay, demand spike, stock imbalance) as soon as they occur.
- Simulate the downstream impact of a disruption on a digital twin before it plays out in reality.
- Generate and evaluate multiple candidate response actions (reorder, reroute, reallocate stock) by testing each inside the twin.
- Recommend the best action with a clear, quantified justification (cost, delay, service-level impact).
- Execute the approved action and update both the twin and system state, closing the loop.

3. System architecture

The system is five stacked layers. Live state flows upward from the data layer through the twin and agents to the dashboard; approved actions flow back down from the dashboard through the backend and agents into the twin.



Layer-by-layer breakdown

Layer	Responsibility	Key components
Data layer	Provides realistic starting data and disruption events for the demo.	Kaggle DataCo SCM dataset; synthetic disruption event generator
Digital twin layer	Holds the live, simulatable state of the whole supply chain.	SimPy discrete-event simulation engine; state model for warehouses, shipments, orders, suppliers
Agent layer	Detects problems, tests responses, and decides what to do.	Monitoring/Exception agent, Simulation agent, Decision/Recommendation agent, Orchestrator agent
Backend / API layer	Glues everything together and exposes it to the browser.	FastAPI, REST endpoints, WebSocket for live push updates
Frontend / dashboard layer	Where the human sees state, tests what-ifs, and approves actions.	Live dashboard, what-if simulation panel, approval UI

Core agent loop

Every disruption — whether injected manually for a demo or arising from the simulated data — runs through the same seven-stage loop:



- Perceive — an agent reads the current twin state (inventory levels, in-transit shipments, supplier status).
- Detect — the Monitoring/Exception agent flags anomalies: a delay signal, a demand spike, a stock imbalance.
- Simulate — the Simulation agent generates 2–3 candidate actions (reorder, reroute, reallocate) and fast-forwards each through the twin.
- Recommend — the Decision/Recommendation agent compares projected outcomes and picks the best one, with a plain-language reason.
- Approve — a human planner reviews the recommendation on the dashboard and approves or rejects it.
- Act — on approval, the Orchestrator commits the action to the twin's live state (and, in a future real deployment, to the ERP).
- Log — the disruption, every candidate considered, and the decision taken are written to an audit trail, which is what closes the loop back into monitoring.

4. Detailed agent specifications

Four agents are needed. Each should be a small, focused unit with a clear input/output contract — don't let any one agent try to do two jobs.

4.1 Monitoring / Exception agent

- Input: a stream (or polling snapshot) of twin state — inventory levels, shipment ETAs, supplier lead-time updates.
- Logic: rule-based thresholds plus an LLM reasoning pass — e.g. flag when stock will breach safety level within N days given current demand, or when a shipment's delay exceeds X% of its planned lead time.
- Output: a structured “disruption event” object (type, affected node, severity, detected-at timestamp).
- Design tip: keep the anomaly rules deterministic and cheap (no LLM call needed to just compare numbers to thresholds); reserve the LLM call for turning the raw anomaly into a readable description.

4.2 Simulation agent

- Input: a disruption event plus the current twin state.
- Logic: generates 2–3 candidate response actions (e.g. “reorder from Supplier B”, “reroute via Warehouse C”, “wait and absorb”), then runs each forward through the SimPy twin in a forked, fast-forwarded simulation — never on the live state.
- Output: for each candidate, a projected outcome: cost delta, delivery delay, stockout risk, service-level impact.
- Design tip: this agent's core job is orchestrating simulation runs, not reasoning — keep the LLM's role limited to proposing which actions to try; let SimPy do the actual forecasting.

4.3 Decision / Recommendation agent

- Input: the set of candidate actions and their simulated outcomes from the Simulation agent.
- Logic: compares outcomes against a configurable objective (e.g. minimise cost subject to a stockout-risk ceiling) and picks the best candidate.
- Output: the chosen action plus a plain-language justification with the quantified numbers, e.g. “saves ₹40k, reduces stockout risk by 60%, adds 1 day of delivery delay.”

- Design tip: this is the one agent where LLM reasoning genuinely adds value over a hardcoded rule — use it to weigh trade-offs and phrase the justification, but keep the underlying numbers coming from the simulation, never invented by the model.

4.4 Orchestrator agent

- Input: nothing external — it drives the other three.
- Logic: sequences Perceive → Detect → Simulate → Recommend → (human approval) → Act → Log; owns retries, timeouts, and error handling between steps.
- Output: commits the approved action to the twin's live state and triggers the audit log entry.
- Design tip: build this with LangGraph (a state graph) or CrewAI (a task/crew model) rather than hand-rolled if/else — you get built-in state passing and easier debugging of where in the pipeline something failed.

5. Tech stack

Component	Tool	Notes
Digital twin / simulation	SimPy (discrete-event simulation)	Models warehouses, shipments, and suppliers as a network of states that update on events
Agent orchestration	LangGraph or CrewAI	LangGraph if you want explicit state-graph control; CrewAI if you want a faster-to-set-up role/task model
LLM reasoning core	Claude or GPT-family	Used for anomaly description, action proposal, and recommendation justification
Forecasting (demand signal)	Prophet or simple ARIMA	Feeds expected demand into the twin so disruptions play out realistically
Frontend	React (or Next.js)	Custom dashboard with live visualization and animation, not a generic admin template
Backend	FastAPI (Python)	Serves REST + WebSocket APIs; the single glue layer between twin, agents, and frontend
Data	Kaggle DataCo Supply Chain dataset + synthetic event generator	Initializes realistic demand, lead-time, and cost patterns

6. Dashboard UI/UX — what to actually build

The dashboard is the single most “demo-able” part of this project, so it's worth specifying precisely rather than leaving it generic.

6.1 Live dashboard (home screen)

- A network / map-style view of warehouses and in-transit shipments — nodes for warehouses (colored by stock health: green/amber/red), lines for shipments in transit (colored by on-time/delayed status).
- A side panel listing current inventory levels per warehouse, with a small trend sparkline per SKU.
- A “inject disruption” button (for demo purposes) that lets you manually trigger a delay, spike, or shortage and watch the dashboard react live.
- A live activity feed showing agent actions as they happen (“Monitoring agent flagged a delay on Shipment #482...”) — this is what makes the multi-agent pipeline visible to an audience instead of feeling like a black box.

6.2 What-if simulation panel

- A form where the planner picks a scenario (“reroute via Warehouse B”, “place a rush order now”) and hits “simulate”.
- Results shown as a before/after comparison: three or four key metrics (cost, delay, stockout risk, service level) shown as simple bar or delta indicators — not raw JSON.
- This panel should feel instant even though it's running a simulation — show a short loading state (1–2 seconds) rather than blocking silently.

6.3 Approval UI

- When the Decision agent produces a recommendation, it appears as a card: the action, the plain-language reason, and the three headline metrics, with Approve / Reject buttons.
- On approve, the dashboard should visibly update in real time (via the WebSocket connection) — the affected warehouse/shipment node changes color or state immediately.
- Keep a running “decisions timeline” so a viewer (or your professor) can see the full history of disruptions and how they were resolved without digging through logs.

6.4 Visual design notes

- Use a clean, flat design — no heavy gradients or 3D effects; a network diagram with clear color-coded status reads better on a projector than a busy dashboard.
- Pick one accent color for “normal” and one or two for “warning/critical” — consistency here does more for readability than any individual widget.
- Animate state changes (a shipment moving, a warehouse level shifting) with simple CSS transitions — it's what sells the “live twin” feeling in a demo.

7. Shared schemas

7.1 Twin state (example)

```
{ "warehouses": [{ "id": "WH-01", "location": "Mumbai", "inventory": { "SKU-100": 420 },  
"safety_stock": { "SKU-100": 150 } }], "shipments": [{ "id": "SHP-482", "origin": "WH-01",  
"destination": "WH-03", "eta": "2026-07-12", "status": "in_transit", "delay_days": 0 }], "suppliers":  
[{ "id": "SUP-07", "lead_time_days": 9, "reliability_score": 0.86 } ] }
```

7.2 Disruption event (example)

```
{ "event_id": "EVT-118", "type": "shipment_delay", "affected_id": "SHP-482", "severity": "medium",  
"detected_at": "2026-07-08T10:32:00Z", "description": "Shipment SHP-482 is projected 3 days late" }
```

7.3 Action / recommendation (example)

```
{ "action_id": "ACT-054", "event_id": "EVT-118", "chosen_action": "reroute_via_WH-02",  
"candidates_considered": 3, "justification": "Saves ₹40k, reduces stockout risk by 60%, adds 1 day  
delivery delay", "status": "pending_approval" }
```

8. Team split — 4 members, equal contribution

Each teammate owns one vertical slice end-to-end, which keeps contribution genuinely equal — nobody is “just the frontend person” or “just doing setup.” Each slice has real design decisions, real code, and a visible piece of the final demo.

Member A — Digital twin

- Build the SimPy simulation engine: warehouses, in-transit shipments, supplier lead times as a state machine that updates on events (order placed, shipment delayed, demand realized).
- Build the synthetic disruption injector used for the demo and for testing.
- Load and shape the Kaggle DataCo dataset into the twin's initial state.
- Own the twin state schema (Section 7.1) and keep it in sync with what the other three members need.

Member B — Agents

- Build all four agents: Monitoring/Exception, Simulation, Decision/Recommendation, and the Orchestrator.
- Choose and set up LangGraph or CrewAI; design the prompts for anomaly description and recommendation justification.
- Develop against a stubbed/mocked twin state early on, so this doesn't block on Member A finishing the twin.
- Own the disruption event and action/recommendation schemas (Sections 7.2, 7.3).

Member C — Backend / API

- Build FastAPI endpoints that expose twin state, trigger the agent pipeline, and accept approval/rejection actions.
- Build the WebSocket layer that pushes live state updates to the frontend as the twin and agents change it.
- Act as the integration owner — enforce the shared schemas from Section 7 across the other three modules and catch drift early.
- Write the audit-log persistence (even a simple SQLite table is enough for the demo).

Member D — Frontend / dashboard

- Build the React dashboard: live network view, what-if panel, and approval UI as specified in Section 6.
- Wire up the WebSocket client for real-time updates.
- Build against mocked WebSocket data before the backend is live, so this also doesn't block on Member C.
- Own the demo script and visual polish — this is the module a professor or audience will actually watch.

9. Week-by-week methodology

Week	All 4 together	Individual focus
1	Agree on the twin state, event, and action schemas (Section 7). Pick and download the Kaggle DataCo dataset. Set up the shared repo, branching strategy, and a docker-compose dev environment.	Everyone reads the relevant part of the tech stack docs for their module.
2–3	Short daily/every-other-day sync to catch schema drift early.	Each member builds their module in parallel against stubs/mocks of the other three (Section 8).
4	First integration checkpoint: connect twin ↔ backend only, verify state reads/writes work.	Continue building; Member B and D keep developing against mocks.
5	Full integration: backend ↔ agents ↔ frontend all connected and talking over the real APIs and WebSocket.	Fix integration bugs together; this week is naturally collaborative.
6	Define 4–5 disruption scenarios together (Section 10). Run each through the agentic pipeline and the static baseline.	Each member instruments their module for the metrics needed (cost, delay, stockout rate, response time).
7	Polish, bug fixes, and demo rehearsal. Script exactly which disruption to inject live and in what order to click through the dashboard.	Each member polishes their own module's visual/UX details.

10. Evaluation plan

Your strongest result for the professor is a direct, quantified comparison between the agentic system and a static rule-based baseline (e.g. “reorder automatically when stock falls below a fixed threshold”) run on the same disruption scenarios.

Suggested scenarios

- A single supplier delay of a fixed number of days on a high-volume SKU.
- A sudden demand spike (e.g. +40%) on one warehouse.
- A shipment lost or significantly delayed in transit.
- A compound scenario: a supplier delay that coincides with a demand spike.
- A low-severity disruption, to show the system doesn't overreact to minor noise.

Metrics to report

Metric	What it shows
Stockout rate	How often inventory actually ran out under each approach
Total cost	Rush-order premiums, holding costs, and lost-sales costs incurred
Average response time	Time from disruption occurring to a corrective action being taken
Service-level impact	Percentage of orders fulfilled on time

11. Deliverables checklist

- Working live demo: inject a disruption → twin updates → agents detect, simulate, and recommend → human approves → metrics visibly improve.
- Quantified agentic-vs-baseline comparison across the 4–5 scenarios (Section 10).
- Source code, organized by the four modules in Section 8, with the shared schemas documented.
- Short slide deck or write-up covering problem, architecture, methodology, and results — reuse the architecture and pipeline diagrams from Sections 3 and this document.
- A “future work” note on extending the simulation environment to a real ERP integration.

12. Risks and mitigations

Risk	Mitigation
Schema drift between modules causes late integration pain	Freeze schemas in week 1; Member C (backend owner) enforces them across all API calls
Agent LLM calls are slow or flaky during a live demo	Cache/pre-warm responses for the specific scenarios you'll demo; always have a recorded backup video
SimPy twin doesn't behave realistically enough to trust the numbers	Calibrate initial state against the Kaggle dataset's real demand/lead-time distributions before building agents on top
One module runs late and blocks integration	Everyone builds against mocked stubs of the other three modules from week 1, so no one is truly blocked